

Universidade Federal do Rio Grande do Norte
Escola de Ciências e Tecnologia
IX Semana de Ciências e Tecnologia
Minicurso de Introdução a Banco de Dados

SQL e PostgreSQL

Parte 04

Breno Santana Santos
⟨breno.santos@ufrn.br⟩

12 de novembro de 2026

Sumário

1. Relacionamentos entre Tabelas
   - Um para um (1:1)
   - Um para muitos (1:n)
   - Muitos para muitos (n:n)

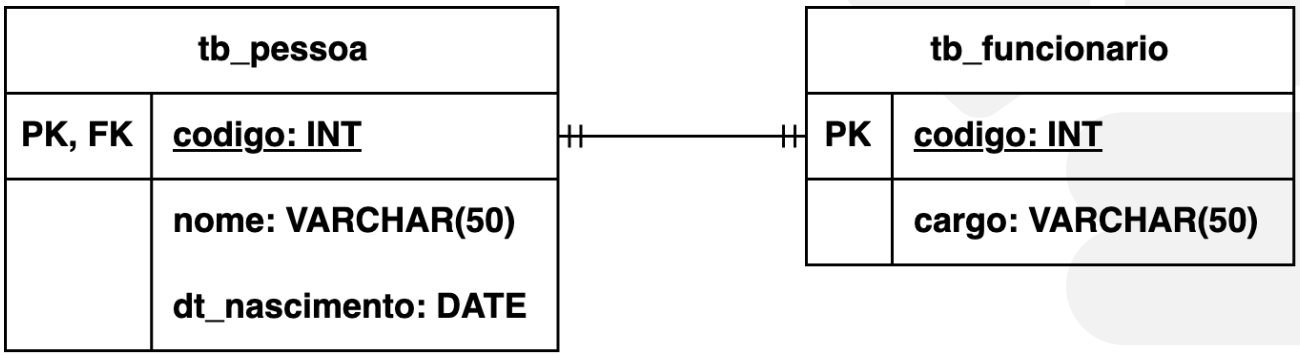
Relacionamentos entre Tabelas

- Por meio de chaves estrangeiras, é possível criar relacionamentos entre tabelas.
- Sempre que duas tabelas (entidades) apresentarem interdependência, existe um relacionamento entre elas.
- Os relacionamentos entre tabelas podem ser:
  - Um para um (1:1);
  - Um para muitos (1:n);
  - Muitos para muitos (n:n).

Relacionamentos entre Tabelas

Um para um (1:1)

- Ocorre sempre que uma tabela possui uma instância associada a, no máximo, uma instância da outra tabela.
- Esse tipo de relacionamento é considerado efetivamente raro.
- Para definir esse tipo de relacionamento, podemos utilizar duas estratégias:
  - Uso de chaves estrangeiras;
  - Unificar os atributos em uma única tabela.
- **Exemplo:**



- **Exemplo:**

```
CREATE TABLE tb_pessoa (
   codigo SERIAL NOT NULL PRIMARY KEY,
   nome VARCHAR(50) NOT NULL,
   dt_nascimento DATE NOT NULL
);

CREATE TABLE tb_funcionario (
   codigo SERIAL NOT NULL PRIMARY KEY,
   cargo VARCHAR(50) NOT NULL,
   FOREIGN KEY (codigo) REFERENCES
     tb_pessoa (codigo)
);
```

- **Exemplo:**

```
CREATE TABLE tb_funcionario (
   codigo SERIAL NOT NULL PRIMARY KEY,
   nome VARCHAR(50) NOT NULL,
   dt_nascimento DATE NOT NULL,
   cargo VARCHAR(50) NOT NULL
);
```

Relacionamentos entre Tabelas

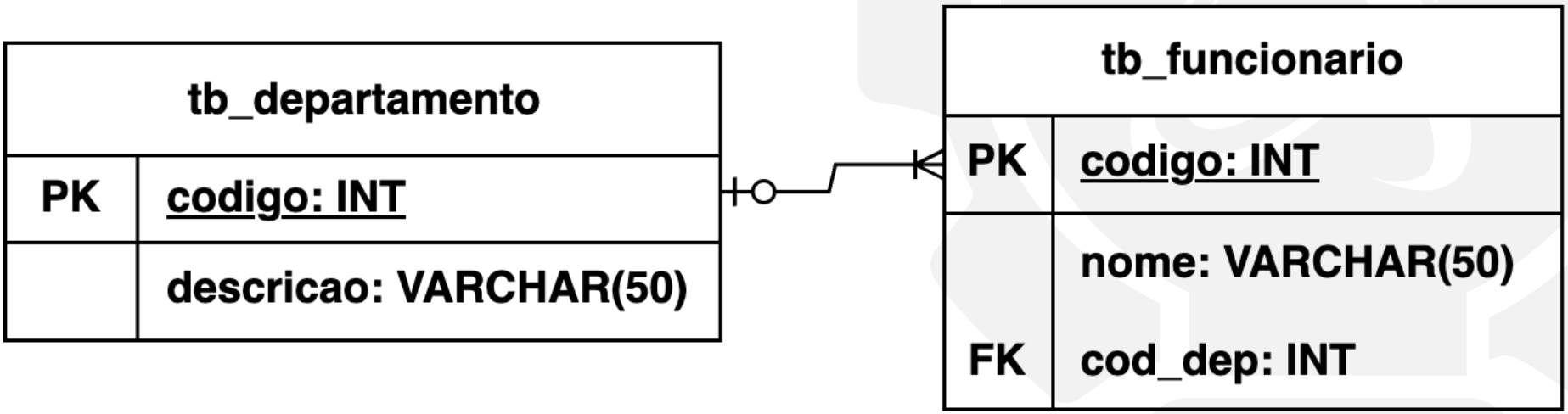
Um para muitos (1:n)

- Ocorre sempre que uma tabela está associada a um ou mais registros de outra tabela e esta se relaciona, no máximo, com um registro da primeira tabela.
- Esse tipo de relacionamento é o mais comum e simples de entender.
- Na entidade de cardinalidade 1, são mantidos os dados básicos. Já, na entidade de cardinalidade muitos, é determinada a lista de atributos não chave da tabela-filha.
- Para definir esse tipo de relacionamento, deve-se incluir a chave estrangeira na tabela de multiplicidade muitos, em que a chave estrangeira irá referenciar a chave primária da tabela de multiplicidade 1.

Relacionamentos entre Tabelas

Um para muitos (1:n)

- **Exemplo:**



- **Exemplo:**

```
CREATE TABLE tb_funcionario (
   codigo SERIAL NOT NULL PRIMARY KEY,
   nome VARCHAR(50) NOT NULL,
   cod_dep INT NOT NULL,
   FOREIGN KEY (cod_dep)
   REFERENCES tb_departamento (codigo)
);

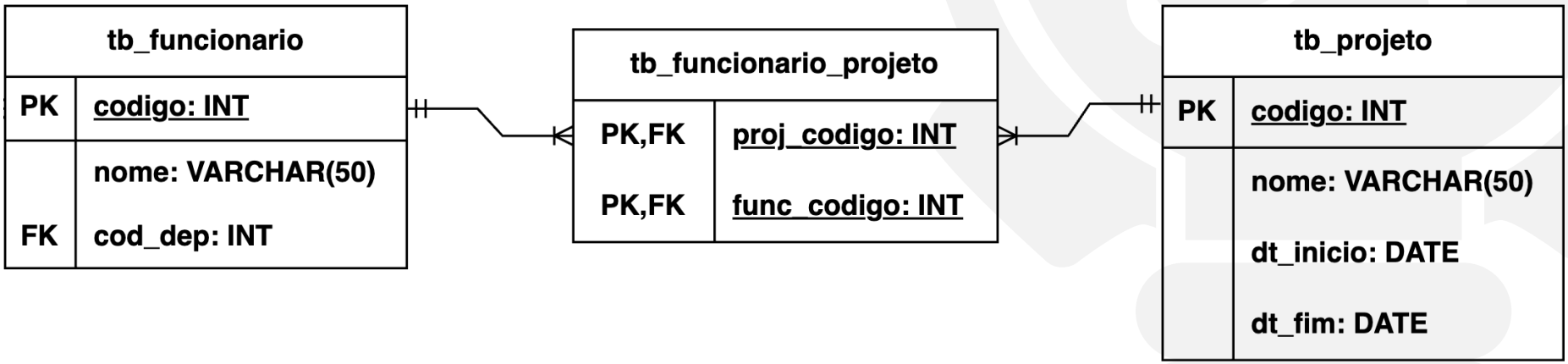
CREATE TABLE tb_departamento (
   codigo SERIAL NOT NULL PRIMARY KEY,
   descricao VARCHAR(50) NOT NULL
);
```

Relacionamentos entre Tabelas

Muitos para muitos (n:n)

- Ocorre sempre que uma tabela se relaciona com vários registros de outra tabela e esta, por sua vez, relaciona-se com vários registros da primeira tabela.
- Esse tipo de relacionamento é transformado em dois relacionamentos de um para muitos (1:n) e uma terceira tabela é criada como intermediária entre esses relacionamentos.
- A tabela intermediária é chamada de tabela de relacionamento.
- Cada tabela-base se relacionará com a tabela intermediária por meio de relacionamento de um para muitos, sendo que a cardinalidade muitos pertence à tabela intermediária, enquanto que, a cardinalidade 1 pertence às tabelas-base.

- A chave primária dessa tabela intermediária é uma chave primária composta, formada pelas chaves primárias das tabelas-base, bem como a chave primária da tabela intermediária faz referência às chaves primárias das tabelas base.



- **Exemplo:**

```
CREATE TABLE tb_funcionario (
   codigo SERIAL NOT NULL PRIMARY KEY,
   nome VARCHAR(50) NOT NULL,
   cod_dep INT NOT NULL,
   FOREIGN KEY (cod_dep)
   REFERENCES tb_departamento (codigo)
);

CREATE TABLE tb_projeto (
   codigo SERIAL NOT NULL PRIMARY KEY,
   nome VARCHAR(50) NOT NULL,
   dt_inicio DATE NOT NULL,
   dt_termino DATE NOT NULL
);
```

Relacionamentos entre Tabelas

Muitos para muitos (n:n)

```
CREATE TABLE TB_FUNCIONARIO_PROJETO (
  proj_codigo INT NOT NULL,
  func_codigo INT NOT NULL,
  PRIMARY KEY (proj_codigo, func_codigo),
  FOREIGN KEY (proj_codigo)
    REFERENCES tb_projeto (codigo),
  FOREIGN KEY (func_codigo)
    REFERENCES tb_funcionario (codigo)
);
```